Informe del Proyecto: Seguidor de Luz Solar

Ignacio Benjamin Ceballos

En este proyecto se desarrolla el software de control para un pequeño sistema, compuesto por 2 sensores de luz con fotoresistencias, y un servomotor. Buscamos que pueda orientarse en dirección a una fuente de luz intensa, emulando el comportamiento de un seguidor solar.

Introducción

El objetivo principal es aplicar los conceptos aprendidos sobre microcontroladores, lectura de la señales de salida de los sensores analógicos,interrupciones,utilidad de los temporizadores y modulación por ancho de pulso (PWM) . Haciendo uso de la placa STM32F411E-DISCO y el entorno de desarrollo STM32Cube MX.

La idea general para el funcionamiento es detectar diferencias de iluminación entre las dos fotorresistencias y corregir su orientación moviendo el servomotor para seguir la luz.

Implementación:

Obtención de las mediciones de los sensores

Para esta parte, primero hay que comprender el conversor analógico digital (ADC). Este permite, como su nombre lo indica, transformar la señal analógica, en este caso proveniente del sensor, a una señal digital. Entonces el microcontrolador tiene ciertos pines asignados para entradas analógicas, y mediante ellos se puede hacer uso del conversor.

El ADC de esta placa divide los canales que puede medir en dos grupos que pueden funcionar en simultáneo o independientes.

- Grupo Regular: Es usado para adquisición general de datos, útil para medir señales que cambian lentamente o para mediciones con temporización predecible.
- Grupo Inyectado:Se utiliza para conversiones rápidas y asíncronas, útil cuando se necesitan respuestas de baja latencia o cuando se necesita medir múltiples canales juntos y nos interesa que los datos sean del “mismo instante”. Puesto que estos canales hacen las lecturas con mayor prioridad.

Modos:

  - **Single conversion:** Hace una sola conversión.
  - **Continuous conversion**(Grupo Regular)**:** Es útil cuando se necesita muestrear una señal analógica constante, en este modo el adc comienza una nueva conversión apenas termina una. Se acostumbra a utilizar junto con la DMA(Direct memory access), que permite que las conversiones continuas se almacenen automáticamente en memoria sin intervención del CPU.

- **Discontinuous conversion:** Puede ser usado para convertir una secuencia corta de n conversiones (n ≤ 8) . Es decir que se seleccionan un número reducido de canales.
- **Scan:** Permite seleccionar varios canales analógicos configurados y realizar la conversión uno tras otro sin tener que comenzar una conversión nueva para cada uno.

Las conversiones se pueden disparar con temporizador o por software. En esta implementación la lectura de los sensores de luz S1 y S2 se realizó mediante el ADC usando Canales Inyectados en modo scan para leer los canales Rank 1 y Rank 2. Disparando la lectura por software.

Hay diferentes formas en las que el cpu puede coordinar y esperar los resultados de un dispositivo lento (el ADC en este caso, puesto que es el que demora en hacer la conversión de la salida de los sensores) antes de continuar con la siguiente tarea.

A estas se les llama estrategias de sincronización.

- Polling: El CPU consulta periódicamente el estado del ADC para saber si la conversión terminó y así leer el dato. Es sencillo pero ineficiente porque el CPU puede estar bloqueado esperando.
- Interrupciones: El ADC genera una interrupción al completar la conversión, el CPU atiende la interrupción y lee el dato. Permite que el CPU realice otras tareas mientras espera, es más eficiente.

en este caso se usó:

Polling:

Para la sincronización, se implementó un método de polling dentro del bucle principal:

1. Se inicia la conversión inyectada con el servicio HAL_ADCEx_InjectedStart(&hadcX).
2. Se utiliza la función HAL_ADCEx_InjectedPollForConversion(&hadcX, timeout) para esperar activamente (haciendo *polling*) a que la bandera **JEOC**(Injected End Of Conversion) se active, lo que indica que ambos canales han completado la medición.
3. Una vez finalizada la conversión, se recuperan las lecturas de los registros JDRx mediante el servicio HAL_ADCEx_InjectedGetValue(&hadcx,rank) y el valor que obtenemos es la lectura digital del ADC. Como tiene configurada una resolución de 12 bits, proporciona un rango de valores de 0 a 4095 unidades digitales.

Limitación Identificada:

Esta implementación introduce un costo de bloqueo del CPU durante el periodo de conversión. Para una optimización o para proyectos con mayores exigencias de CPU, una estrategia de sincronización mejor sería usar Interrupciones Inyectadas, lo que permite que el CPU realice otras tareas mientras el ADC opera de forma independiente.

Movimiento del motor

Para poder controlar el servomotor necesitamos comprender cómo funciona la modulación por ancho de pulso(PWM).PWM es una técnica utilizada para controlar la cantidad de energía entregada a un dispositivo sin variar el voltaje, sino modulando el ciclo de trabajo de una señal cuadrada. Esta es la manera que tenemos de comunicarnos con el servomotor.

El servomotor utilizado en este proyecto se controla mediante pulsos PWM con las siguientes características:

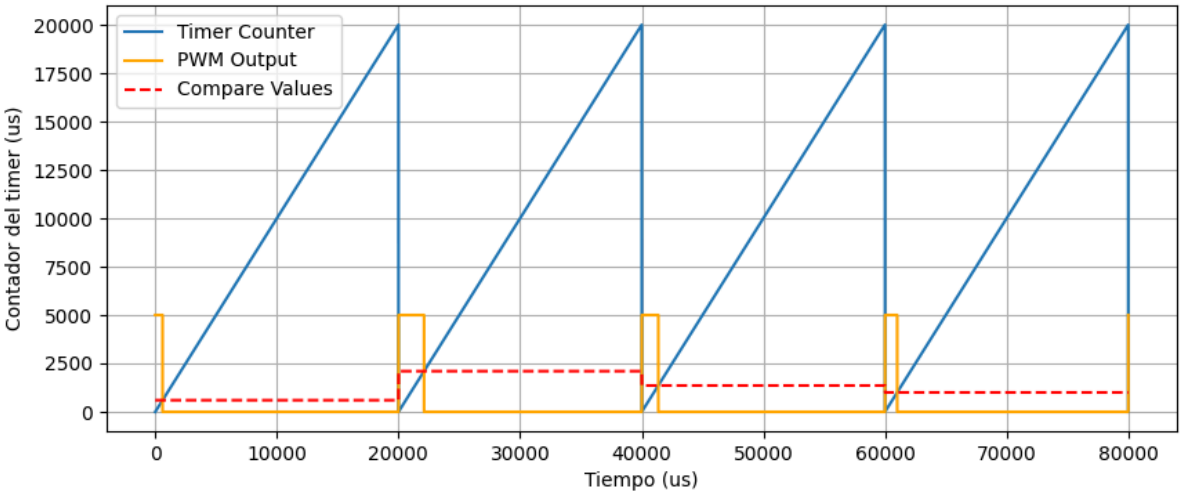
- Periodo T (Tiempo que tarda en completarse un ciclo completo de la señal): 20 ms.

- Frecuencia f de 50 Hz (número de ciclos por segundo f = 1/T).

- El rango de operación es de aproximadamente 180°, determinado experimentalmente mediante la observación de los límites físicos al aplicar pulsos entre 600µs y 2100µs. Estos valores pueden variar según el modelo específico de servomotor.

Es decir que tenemos anchos de pulsos que van de 0.6 ms a 2.1 ms.

Así que cuando se envía un pulso que dura 0.6 milisegundos estaremos ordenando que esté en aproximadamente 0° y cuando mandamos uno que dura 2.1 milisegundos le decimos que vaya a aproximadamente 180°.
Ahora queda ver como se hace para enviar estos pulsos de manera precisa cada 20 milisegundos. Para ello se configura un temporizador con un periodo de 20000 µs (20 ms).



(Aclaración: el output de la señal pwm en realidad es en binario, al estar en alto es 1, sino 0)

En este gráfico tenemos los valores que nos da el timer, y seteamos un valor de comparación con respecto al valor del timer, en este caso los valores de comparación son 600, 2100, 1350, 1000 en el mismo orden que se ven en el gráfico. Enviamos la señal al reiniciarse el conteo del temporizador y dejamos de enviar cuando llega al valor que seteamos con cmp, y de esta manera mandamos la señal del tiempo que necesitamos y respetamos el tiempo que el servomotor requiere entre señales para funcionar correctamente.

En la implementación hacer esto es sencillo, puesto que podemos configurar un canal del temporizador para que genere una señal pwm, así que solo tenemos que preocuparnos por setear los valores de comparación que queramos para ajustar la posición. Esto en el código se hace con la directiva __HAL_TIM_SET_COMPARE(&htimX,TIM_CHANNEL_X,valor_deseado).Esta directiva modifica el registro capture compare X del timer X seleccionado.

Algoritmo de seguimiento

El algoritmo implementado sigue estos pasos:

1. Lectura de ambos sensores (S1 y S2)
2. Cálculo de la diferencia de intensidad de la luz: Δ = S1 - S2
    - Si Δ >0 (S1>S2), el motor debe ir hacia S1
    - Si Δ <0 (S2>S1), el motor debe ir hacia S2
3. Ajuste del nuevo valor: nuevo_valor = valor_actual ± Δ * K
    - (Δ * K se resta o suma dependiendo de la posición física de S1 y S2)
    - K es un factor de escalamiento que convierte a Δ en una acción de control (como el “número de pulsos que debe moverse”).
    - K cumple el trabajo de escalar la entrada y convertir la unidad al mismo tiempo.
4. Actualización de la posición del servomotor con nuevo_valor **(*)**
5. Repetición continua del proceso

(*)En esta parte es necesario agregar un retraso de unos milisegundos para darle tiempo al servomotor a alcanzar físicamente la nueva posición y que no se lea el sensor antes de que se llegue a la posición nueva. La forma más eficiente de hacerlo es con otro temporizador que tome esos milisegundos y levante una interrupción en el momento en que haya pasado el tiempo, para no tener el cpu ocupado haciendo nada.